

Senior AI/CV + Big-Tech Readiness Master Learning Plan (v5)

Senior AI/CV Engineer + Big-Tech Readiness — Master Learning Plan (v5, two-block)

Owner: MD Hashim · **Window:** ~mid-Aug 2026 → end-Aug 2027 (~13 months) · **Total effort:** ~1,730 focused hours · **Structure:** two blocks + two muscle-memory threads

The program at a glance

This version extends the 7-month depth plan with a second ~6-month block for breadth, GenAI/agents, and big-tech interview readiness — and adds long-running "muscle-memory" threads (DSA and C++) that build the way such skills actually stick: little and often, over many months.

Block 1 — Depth & foundations (Aug 2026 → mid-Mar 2027, ~7 months, ~1,140 hrs). Unchanged: a rapid fundamentals brush-up, then bottom-up mastery — math foundations → your medical-CV domain → DL theory → quantum → production → specialization. Runs at ~38–40 hrs/week to hit the mid-March cap without cutting depth.

Block 2 — Breadth + big-tech readiness (mid-Mar → Aug 2027, ~5.5 months, ~500 hrs + interviewing). GenAI & agentic AI first (as you asked), then a computer-vision *expert* layer (classical/geometric/3D vision + C++/edge deployment for your hardware-integration need), then a big-tech interview crescendo (DSA, system design, behavioral). Runs a touch lighter (~30–34 hrs/week) to leave real capacity for interview loops.

Thread T (parallel, from ~Month 3). DSA and C++ are muscle-memory skills — they reward a daily ~40-minute ritual sustained over ~10 months far more than a short cram. So a light DSA + C++ thread starts once your foundations are locked in (~Week 9, early Oct 2026) and runs continuously until it feeds the Block 2 interview crescendo.

Why this order, and why the thread starts at Month 3 (not now)

You gave me the call on when to start DSA/C++, and the smart move is **Month 3, not today** — for two reasons. First, Block 1's opening (the Phase 0 brush-up + the Phase 1 math foundations) is the highest-leverage, most focus-hungry stretch of the whole program; diluting it with a second daily discipline is a bad trade. Second — the point you already intuited — DSA and C++ are *muscle memory*: what builds them is **consistency over a long horizon**, not front-loaded volume. A 40-min/day ritual from October to July is ~10 months of spaced repetition, which beats any short intensive block and peaks you right as interviews start. GenAI/agents and the CV-expert layer sit in Block 2 because they're breadth on top of the Block-1 core; the interview crescendo comes last so DSA/system-design skills peak exactly when you begin interviewing.

Timeline & weekly load

Block / thread	Window	Weekly load	Content
Block 1 (Phases 0–6)	mid-Aug 2026 → mid-Mar 2027	~38–40 hrs/wk	Depth: foundations → CV/segmentation → DL theory → quantum → production → specialization
Thread T (DSA + C++)	early Oct 2026 → mid-Jul 2027	+~4 hrs/wk (daily ritual)	Muscle memory: C++ basics → DSA-in-C++, sustained
Block 2 (Phases 7–9)	mid-Mar → Aug 2027	~30–34 hrs/wk	Breadth: GenAI/agents → CV expert → interview crescendo

On the combined load: during Block 1's second half the thread pushes you to ~42–44 hrs/week. That is a lot on top of a full-time role. The thread is deliberately the *lightest* thing on your plate (a short daily ritual, often a welcome change of pace from heavy theory) and it is the **first thing to dial back** — to 2–3 hrs/week — if any week gets overwhelming, since its value is consistency, not volume. Treat the self-tests as the real gates, protect one rest block a week, and bank buffer where you can. If the calendar and your bandwidth genuinely collide, tell me and I'll rebalance rather than let you burn out.

Phase summary (the whole plan at a glance)

Block 1 — Depth & foundations · ~1,140 hrs · ~38-40 hrs/wk

Phase	Focus	Effort	Weeks	Approx. dates
0	Rapid brush-up — stats, ML & DL fundamentals	~45 hrs	1-1.5	mid-late Aug 2026
1	Foundations — mathematics + neural nets from first principles	~190 hrs	2-6	late Aug → early Oct
2	Applied CV & Medical Image Segmentation (active domain)	~185 hrs	7-11	early Oct → early Nov
3	Deep Learning theory & modern architectures	~150 hrs	12-15	Nov → early Dec
4	Quantum Computing & QML	~275 hrs	16-22	Dec → late Jan
5	Full-stack / production AI systems	~160 hrs	23-26	late Jan → mid-Feb
6	Specialization + integration	~135 hrs	27-30	mid-Feb → mid-Mar 2027

Thread T — DSA + C++ muscle memory · ~4 hrs/wk · Weeks 9 → 47 (starts early Oct 2026; see its own section below)

Block 2 — Breadth + big-tech readiness · ~500 hrs + interviewing · ~30-34 hrs/wk

Phase	Focus	Effort	Weeks	Approx. dates
7	GenAI & Agentic AI	~150 hrs	31-35	mid-Mar → mid-Apr 2027
8	Computer Vision Expert (+ C++/edge deployment)	~200 hrs	36-42	mid-Apr → early Jun
9	Big-Tech Interview Crescendo — DSA, system design, behavioral	~150 hrs	43-47	Jun → mid-Jul
—	Active interviewing + DSA/design maintenance + buffer	—	48-55	mid-Jul → Aug 2027

Grand total ≈ 1,730 focused hours across ~13 months. Specialized math (optimization, information theory, group theory beyond the Phase 1 core) is threaded into the phase that uses it; its hours are included above.

Thread T — DSA + C++ (the muscle-memory thread)

~4 hrs/week · a daily ~40-min ritual · Weeks 9 → 47 (early Oct 2026 → mid-Jul 2027)

Runs in parallel with the phases as a short daily warm-up, not a block. Durable fluency comes from repetition, so consistency matters more than any one session's length. Do it first thing, before the day's phase work, while your head is fresh.

Stage T1 — C++ fundamentals (Weeks 9-16, ~8 wks, ~32 hrs). Get comfortable with modern C++ so you can then solve problems in it: types, references/pointers, RAII, the STL (vector, map, set, algorithms), classes, templates, move semantics, smart pointers. This directly serves your hardware-integration need and sets up DSA-in-C++.

- **learncpp.com** — the best free comprehensive C++ course; work sections 1-17 across these weeks — <https://www.learncpp.com/>
- **The Cherno — C++ series** (YouTube; the *why* behind C++ mechanics) — <https://www.youtube.com/@TheCherno>
- Reference: <https://en.cppreference.com/>

Stage T2 — DSA in C++, sustained (Weeks 17-47, ~7 months; ~55 hrs foundational, then rolled into the Phase 9 push). Solve problems daily, in C++, following a pattern-based curriculum rather than random grinding — ~3-4 problems/week, spaced and revisited.

- **NeetCode** — the pattern roadmap + video explanations; work the NeetCode 150 across the thread — <https://neetcode.io/>
- **LeetCode** — the problem bank (do NeetCode's list here) — <https://leetcode.com/>
- **MIT 6.006 — Introduction to Algorithms** (theory under the patterns: complexity, sorting, graphs, DP, hashing) — <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/>
- **Abdul Bari — Algorithms** (YouTube; clearest algorithm-intuition series) — https://www.youtube.com/@abdul_bari
- Reference text: CLRS, *Introduction to Algorithms*

How it ramps: light and steady through Block 1 (build the reflex), then the **Phase 9 crescendo** intensifies it into

interview-grade timed practice and mock rounds. By interview season you'll have ~10 months of spaced reps behind you — the opposite of a panicked six-week cram.

Thread checkpoint (rolling): by end of Block 1, implement from memory in C++ — dynamic arrays, linked lists, stacks/queues, hash maps, binary trees + BST ops, heaps, graph BFS/DFS — and the standard patterns (two-pointer, sliding window, binary search, backtracking, basic DP). Solve a random LeetCode medium in ~30–35 min.

PHASE 0 — Rapid Brush-Up: Stats, ML & DL Fundamentals

~45 hrs · reactivation for panel readiness · do this first

Goal: Rebuild crisp, *out-loud* command of the fundamentals you learned ~2 years ago, so the opening rounds of a panel — the "explain bias-variance," "why does L1 give sparsity," "what does AUC actually measure" questions — never catch you flat. Reactivation is fast; the aim here is fluent articulation, not re-derivation (that comes in Phase 1). StatQuest is purpose-built for this: video-first, one concept at a time, intuition without dumbing down.

Primary resource for the whole phase — **StatQuest with Josh Starmer**: playlists <https://www.youtube.com/@statquest/playlists> · searchable video index <https://statquest.org/>

0.1 Statistics fundamentals (~12 hrs)

StatQuest *Statistics Fundamentals* playlist: distributions; mean/variance/SD; populations vs samples and estimated parameters; the central limit theorem; sampling; hypothesis testing and p-values (one- vs two-tailed); confidence intervals; R^2 ; covariance & correlation; maximum likelihood; bootstrapping; power analysis. Watch at 1.25–1.5× — you're refreshing, not meeting these for the first time.

Verbal checkpoint: explain p-value, confidence interval, CLT, and MLE each in two sentences without notes.

0.2 Classic machine-learning concepts (~18 hrs)

StatQuest *Machine Learning* playlist: bias-variance tradeoff; train/validation/test and cross-validation; the confusion matrix, sensitivity/specificity, precision/recall, ROC & AUC (and when PR curves are the honest choice); linear & logistic regression (and their assumptions); ridge (L2) vs lasso (L1) regularization and *why* L1 induces sparsity; decision trees, random forests, AdaBoost, and gradient boosting / XGBoost — be able to *explain* the algorithm you use in production daily, not just call `.fit()`; SVMs and the kernel trick; k-means and hierarchical clustering; PCA; naive Bayes; entropy and mutual information.

Verbal checkpoint: explain, cold, how gradient boosting differs from a random forest; why L1 zeros weights and L2 shrinks them; what AUC measures and when PR-AUC is more honest.

0.3 Deep-learning fundamentals (~10 hrs)

StatQuest *Neural Networks* playlist + **3Blue1Brown — Neural Networks** (https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi): perceptron and MLP; why non-linear activations are necessary; backpropagation main ideas; gradient descent; common loss functions; tensors; then CNN and RNN/LSTM basics; overfitting, dropout, batch/layer norm, and vanishing/exploding gradients; transfer learning. (Phase 1's Karpathy series turns this intuition into from-scratch code — here you just want the concepts back on the tip of your tongue.)

0.4 Deliverable — the panel articulation cheat-sheet (~5 hrs)

For every concept above, write **one or two crisp sentences you'd actually say to an interviewer**. This is the artifact that fixes your stated pain. Prompts to answer cold: *Explain bias-variance. Why does L1 give sparse weights but L2 doesn't? What does AUC measure, and when is PR-AUC more honest? How does gradient boosting differ from a random forest? What is the kernel trick? Why do we need non-linear activations? What causes vanishing gradients and how do we fix them? What does dropout do at train vs test time? Why does batch norm help? How do you handle class imbalance?*

Phase 0 self-test (verbal, no notes): rapid-fire — explain 15 of the above in 2–3 sentences each, out loud, as if to a panel. Any shaky answer flags a video to re-watch. Keep the cheat-sheet; it seeds the war-chest doc you'll build through every later phase.

PHASE 1 — Foundations: Mathematics + Neural

Networks from First Principles

~190 hrs · the bedrock — do not rush this

Goal: Build the mathematical maturity your senior transition depends on, and turn it into working neural networks so backprop and "the why" are bone-deep. By the end you can re-derive the core results on paper *and* you understand attention mechanically (via Karpathy's GPT build), which unlocks Phase 2.

1.1 Linear algebra (~68 hrs) — the highest-leverage topic in the entire plan

Underpins both deep learning and quantum. Do it thoroughly.

- **3B1B — Essence of Linear Algebra** (geometric intuition first; watch twice if needed) — ~8 hrs https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab
- **MIT 18.06 — Gilbert Strang** (L1, 3, 5, 6, 9, 10, 14, 15, 16, 21, 22, 25, 29, 30 + problems) — ~35 hrs https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/video_galleries/video-lectures/
- **MIT 18.065 — Matrix Methods (Strang)** — the *applied* view: SVD, PCA, low-rank, gradient descent through a linear-algebra lens; closest to ML reality — ~25 hrs <https://ocw.mit.edu/courses/18-065-matrix-methods-in-data-analysis-sign-al-processing-and-machine-learning-spring-2018/>
- Reference kept open: **Matrix Cookbook** — <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf>

Whiteboard checkpoint: derive SVD from eigendecomposition; explain geometrically what a rank-r approximation does; show $A^T A$ is PSD; derive backprop through a linear layer with matrix calculus.

1.2 Calculus + matrix calculus (~22 hrs)

- **3B1B — Essence of Calculus** — ~8 hrs <https://www.youtube.com/playlist?list=PLZHQObOWTQDMsr9K-rj53DwVRMYO3t5Yr>
- **Matrix Calculus for Deep Learning (Parr & Howard)** — best short article on Jacobians for ML; plus Matrix Cookbook practice — ~14 hrs <https://explained.ai/matrix-calculus/>

1.3 Probability & statistics (~45 hrs)

Underpins loss functions, generative models, uncertainty, and QML. Do a serious pass.

- **Harvard Stat 110 — Joe Blitzstein** (the best intro-probability course; work through conditional expectation, MGFs, key distributions) — ~45 hrs <https://projects.iq.harvard.edu/stat110/youtube>
- Alternative if Blitzstein feels slow: **MIT 6.041 / RES.6-012 — Tsitsiklis** <https://ocw.mit.edu/courses/res-6-012-introduction-to-probability-spring-2018/>

Whiteboard checkpoint: derive MLE for a Gaussian; explain conditional expectation as a projection; state and use Bayes cleanly on a non-trivial example.

1.4 Neural networks from first principles (~55 hrs) — the bridge from math to DL

Non-negotiable even with your experience. Do all 10 videos **with the exercises**, re-implementing each without looking at his code. This is where "the why" lives, and the GPT build gives you attention mechanically — which Phase 2 relies on.

- **Andrej Karpathy — Neural Networks: Zero to Hero** — ~55 hrs Playlist: <https://www.youtube.com/playlist?list=PLAqhlrjKxbuWI23v9cThsA9GvCAUhRvKZ> · Hub: <https://karpathy.ai/zero-to-hero.html> Order: micrograd → makemore 1-5 (incl. BatchNorm internals, manual backprop, WaveNet) → GPT from scratch → tokenizer → reproduce GPT-2.

Phase 1 self-test: hand-derive backprop for a 2-layer MLP; implement micrograd-style autodiff from memory; explain why BatchNorm helps and what breaks without it; whiteboard scaled dot-product attention (you built it — now derive the $\sqrt{d_k}$ scaling).

PHASE 2 — Applied Computer Vision & Medical Image Segmentation

~185 hrs · your active domain — now with full grounding · **PRIORITY: high (your day-to-day work)**

Goal: Own MRI / fluorescence brain segmentation end-to-end — physics → preprocessing → architectures → losses → 3D → uncertainty → evaluation. You arrive here already understanding attention and backprop, so nnU-Net, UNETR, and the math beneath the loss functions land hard. By the end you can whiteboard nnU-Net's design choices cold and defend

each in a project discussion.

2.1 Imaging physics (~30 hrs) — most engineers skip this and it shows in their models

- **Allen Elster — Questions & Answers in MRI** (the reference you'll return to) — ~8 hrs <https://mriquestions.com/index.html>
- **OHBM Educational — MRI physics** playlist — ~8 hrs <https://www.youtube.com/@ohbmonline/playlists>
- **Paul Callaghan — "Magnetic Resonance" (Otago)**, first 4 lectures — ~6 hrs https://www.youtube.com/playlist?list=PLqEMVgX2VgZcG_xR1QRdsuc-X_zEgMx-Z
- **iBiology — fluorescence microscopy** primer — ~8 hrs <https://www.youtube.com/playlist?list=PLF513KEDjY9YBNhwMfX6jMI3PlpdW9TMP>

Deliverable: 1-page note — T1 vs T2 vs FLAIR vs T1CE (tissue appearance, why each exists, artifacts to preprocess: motion, bias field, partial volume); for fluorescence: photobleaching, channel bleed-through, autofluorescence and their segmentation impact.

2.2 CNNs + convolution math + U-Net family + losses (~50 hrs)

- **CS231n 2017 — Justin Johnson** (CNN theory: L5, 9, 11 + notes) — ~18 hrs https://www.youtube.com/playlist?list=P_L3FW7Lu3i5jvHM8ljYj-zLfQRF3EO8sYv · <https://cs231n.github.io/>
- **Convolution arithmetic (Dumoulin & Visin)** — canonical reference for shapes/strides/padding/dilation (2D and 3D) — ~5 hrs <https://arxiv.org/abs/1603.07285> · https://github.com/vdumoulin/conv_arithmetic
- **U-Net family, in order, deriving output sizes by hand** — ~15 hrs U-Net: <https://arxiv.org/abs/1505.04597> · V-Net (3D): <https://arxiv.org/abs/1606.04797> · Attention U-Net: <https://arxiv.org/abs/1804.03999> · **nnU-Net (read twice + supplementary):** <https://www.nature.com/articles/s41592-020-01008-z>
- **Loss functions for medical segmentation** (your 95/5 imbalance) — survey + key losses, then **from-scratch PyTorch implementation** (Dice, GDL, Focal, Tversky, Focal Tversky, Boundary, compound DiceCE; unit-test on synthetic imbalanced data; plot gradients vs class ratio) — ~12 hrs Survey: <https://arxiv.org/abs/2006.14822> · Generalized Dice: <https://arxiv.org/abs/1707.03237> · Focal Tversky: <https://arxiv.org/abs/1810.07842> · Boundary loss: <https://arxiv.org/abs/1812.07032>

Whiteboard checkpoint: draw U-Net from memory with tensor shapes; explain why skip connections work; derive the Dice gradient and its instability on empty masks; explain why nnU-Net's "no new architecture" thesis is profound.

2.3 nnU-Net + MONAI + microscopy-native hands-on (~45 hrs) — the biggest applied block

- **MONAI tutorials** (clone, run, modify the brain-seg notebooks) — ~15 hrs <https://github.com/Project-MONAI/tutorials> · Videos: <https://www.youtube.com/@projectmonai/videos>
- **nnU-Net v2** — read the *source*; it teaches more than the paper — ~8 hrs <https://github.com/MIC-DKFZ/nnUNet> · MONAI integration: <https://github.com/Project-MONAI/tutorials/blob/main/nnunet/README.md> · Kitware comparison: <https://www.kitware.com/developing-custom-3d-medical-image-segmentation-solutions-using-out-of-the-box-pipelines-in-monai/>
- **DigitalSreeni (Python for Microscopists)** — the most microscopy-native channel there is; closer to your fluorescence + volumetric work than any Stanford course. ~6–8 videos, code-along — *~14 hrs* Channel: <https://www.youtube.com/@DigitalSreeni/playlists> · Code: https://github.com/bnsreenu/python_for_microscopists · U-Net playlist: <https://www.youtube.com/playlist?list=PLZsOBAYNTZwbR08R959iCvYT3qzhxvGOE>
- **AI for Medical Diagnosis (Rajpurkar, deeplearning.ai)** — audit free; the brain-MRI U-Net segmentation + evaluation modules are on-point — ~8 hrs <https://www.coursera.org/learn/ai-for-medical-diagnosis>

Deliverable: reproduce nnU-Net on a public brain dataset (BraTS subset or a Medical Segmentation Decathlon task); write up which defaults it chose, *why*, and what you'd change for fluorescence microscopy.

2.4 Transformer-based segmentation + foundation models (~20 hrs)

You understand attention from Phase 1, so these land as applications, not mysteries.

- ViT (re-derive attention if rusty): <https://arxiv.org/abs/2010.11929> — ~3 hrs
- TransUNet / Swin-UNet / UNETR / Swin UNETR — ~9 hrs <https://arxiv.org/abs/2102.04306> · <https://arxiv.org/abs/2105.0537> · <https://arxiv.org/abs/2103.10504> · <https://arxiv.org/abs/2201.01266>
- SAM / MedSAM: <https://arxiv.org/abs/2304.02643> · <https://www.nature.com/articles/s41467-024-44824-z> — ~5 hrs
- MedNeXt (a ConvNet that beats transformers on many medical tasks — keeps you honest): <https://arxiv.org/abs/2303.09975> — ~3 hrs

Whiteboard checkpoint: compare inductive biases of CNN vs ViT vs hybrid — when each wins; role of foundation

models in low-data medical regimes; SAM's promptable design and what MedSAM changes.

2.5 3D, uncertainty, evaluation + capstone (~40 hrs)

- **TorchIO** (3D augmentation done right) — ~10 hrs <https://torchio.readthedocs.io/> · <https://www.youtube.com/playlist?list=PLfXlfDgnRPfIEHFx2N6mlbn-MGFOPiX2>
- **Uncertainty (Yarin Gal — MC Dropout, deep ensembles, evidential)** — ~5 hrs https://www.youtube.com/results?search_query=yarin+gal+uncertainty+deep+learning
- **Metrics Reloaded** (definitive 2024 paper on choosing eval metrics) — ~4 hrs <https://www.nature.com/articles/s41592-023-02151-z>
- **Domain shift in medical imaging (Stanford AIMI)** — ~4 hrs <https://aimi.stanford.edu/education/educational-resources>
- **3D-conv whiteboard exercise:** input (D=64, H=128, W=128, C=4), kernel (3,3,3), stride (1,2,2), padding (1,1,1), 32 out-channels — compute output shape AND param count by hand; repeat for transposed + dilated conv — ~2 hrs
- **PHASE 2 CAPSTONE (~15 hrs)** — notebook + 3-page write-up on your mouse-brain data (or a BraTS subset): (1) preprocessing with physics justification, (2) architecture ablation across 3 architectures, (3) loss-function ablation, (4) uncertainty quantification + failure-mode analysis. The senior-engineer artifact — and exactly what a panel wants to see.

Phase 2 self-test: whiteboard U-Net forward pass + skips with shapes; explain why nnU-Net usually beats fancier architectures (and when it doesn't); for a new dataset, walk through diagnosing bias-field artifacts, class imbalance, label noise, domain shift.

PHASE 3 — Deep Learning Theory & Modern Architectures

~150 hrs · the broad, deep DL backbone

Goal: Generalize beyond what Phase 1-2 taught you: the advanced transformer variants, self-supervised learning, generative/diffusion theory, and the training-dynamics math (optimization + information theory). By the end you can defend architecture and training choices on theoretical grounds.

3.1 Optimization for deep learning (~28 hrs) — threaded here because this is where training theory lives

- **Sebastian Ruder — gradient-descent optimization overview** — ~3 hrs <https://www.ruder.io/optimizing-gradient-descent/>
- **Stanford EE364A — Convex Optimization (Boyd)** — selected lectures for the convex intuitions that even non-convex DL builds on — ~22 hrs <https://www.youtube.com/playlist?list=PL3940DD956CDF0622> · book: <https://web.stanford.edu/~boyd/cvxbook/>
- Optimizer/training-dynamics deep dive (SGD, momentum, Adam, warmup, schedules, second-order intuitions) — ~3 hrs

3.2 Information theory for ML (~18 hrs) — the language of loss functions

- **David MacKay — Information Theory, Inference, and Learning Algorithms, ch 1-6** (entropy, KL, mutual info — underpins cross-entropy, VAEs, diffusion, contrastive learning) — ~18 hrs <https://www.inference.org.uk/itila/book.html>

Whiteboard checkpoint: derive cross-entropy from KL; show NLL minimization = MLE; derive the ELBO; explain mutual information and where it appears in InfoNCE / contrastive losses.

3.3 Transformers in depth + modern variants (~40 hrs)

- **Stanford CS25 — Transformers United** (6-8 talks) — <https://web.stanford.edu/class/cs25/> · https://www.youtube.com/playlist?list=PLoROMvodv4rNijRchCzutFw5ItR_Z27CM
- **Lilian Weng's blog** (attention, transformer, RLHF) — <https://lilianweng.github.io/>
- **The Annotated Transformer** (code + math line by line) — <http://nlp.seas.harvard.edu/annotated-transformer/>
- **Modern tricks** (~30 min each): RoPE (<https://arxiv.org/abs/2104.09864>) · RMSNorm (<https://arxiv.org/abs/1910.07467>) · FlashAttention (<https://arxiv.org/abs/2205.14135>) · GQA/MQA (<https://arxiv.org/abs/2305.13245>) · SwiGLU (<https://arxiv.org/abs/2002.05202>) · MoE / Switch (<https://arxiv.org/abs/2101.03961>)

Whiteboard checkpoint: explain RoPE geometrically; why pre-LN beats post-LN; why LayerNorm not BatchNorm in transformers; compute a transformer block's param count from `d_model`, `n_heads`, FFN ratio.

3.4 Vision transformers + modern ConvNets (~25 hrs)

- ViT lectures (Lucas Beyer) — https://www.youtube.com/results?search_query=lucas+beyer+vision+transformer
- ConvNeXt (<https://arxiv.org/abs/2201.03545>) · DINOv2 (self-supervised, relevant to label-scarce medical work — <https://arxiv.org/abs/2304.07193>) · MAE (<https://arxiv.org/abs/2111.06377>)
- **Hugging Face Computer Vision course** — <https://huggingface.co/learn/computer-vision-course>

3.5 Generative modeling & diffusion (~38 hrs)

Diffusion math underpins the modern field and shows up in medical image synthesis/augmentation.

- **Stanford CS236 — Deep Generative Models (Ermon)** (L1-3, 6-8, 11-13) — <https://www.youtube.com/playlist?list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8>
- DDPM (<https://arxiv.org/abs/2006.11239>) · Yang Song, score-based unification + blog (<https://yang-song.net/blog/2021/score/>) · Lilian Weng diffusion post (<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>)
- **Hugging Face Diffusion course** (units 1-2 hands-on) — <https://huggingface.co/learn/diffusion-course>

Phase 3 self-test: derive the diffusion loss (variational + score-matching views); compare positional encodings (sinusoidal/learned/RoPE/ALiBi); explain the role of self-supervised pretraining in low-data regimes.

PHASE 4 — Quantum Computing & Quantum Machine Learning

~275 hrs · rigorous theory beneath your existing hands-on work — now unhurried

Goal: Match your production VQE/ADAPT-VQE/DMET-VQE experience with real theory. Derive VQE from the variational principle, explain barren plateaus mathematically, defend ansatz choices, and discuss QML's genuine limits honestly (per Schuld's critiques). This is *depth on top of experience* — and because you've now done the ML foundations, QML (really kernel methods + variational optimization) lands far deeper.

4.1 Group theory & symmetry (~22 hrs) — threaded here; also pays off in Phase 6 geometric DL

- **Group theory for physicists** (pick one lecture series) — ~15 hrs https://www.youtube.com/results?search_query=group+theory+for+physicists+lectures
- **Geometric Deep Learning lectures (Bronstein)** — the Lie-group / equivariance angle — ~7 hrs <https://geometricdeeplearning.com/lectures/>

4.2 Quantum-mechanics math: Hilbert spaces, operators, tensor products (~25 hrs)

- **MIT 8.04 — Quantum Physics I (Adams)** (L6-12 if the experimental motivation feels slow) — ~15 hrs https://ocw.mit.edu/courses/8-04-quantum-physics-i-spring-2013/video_galleries/lecture-videos/
- **Quantum Country** (spaced-repetition intro — fast and sticky) — ~6 hrs <https://quantum.country/qcvc>
- (Optional, rigor-minded) Frederic Schuller — Geometric Anatomy of Theoretical Physics — ~4 hrs sampled https://www.youtube.com/playlist?list=PLPH7f_7ZlzxTi6kS4vCmv4ZKm9u8g5yic

Whiteboard checkpoint: define a Hilbert space rigorously; tensor-product two qubit states; explain unitarity, Hermitian operators, why measurement outcomes are eigenvalues; distinguish pure vs mixed states + the density matrix.

4.3 Quantum computing fundamentals: the Watrous / IBM track (~80 hrs)

The best free quantum-information series that exists — John Watrous's four IBM courses.

- **Basics of Quantum Information** — ~25 hrs <https://learning.quantum.ibm.com/course/basics-of-quantum-information> · YouTube: <https://www.youtube.com/playlist?list=PLOFEBzvs-VvrXTMy5Y2lqmSaUjfnhvBHR>
- **Fundamentals of Quantum Algorithms** — ~25 hrs <https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms>
- **General Formulation of Quantum Information** (density matrices, channels, POVMs — matters for noise/error mitigation) — ~18 hrs <https://learning.quantum.ibm.com/course/general-formulation-of-quantum-information>

- **Foundations of Quantum Error Correction** — ~12 hrs <https://learning.quantum.ibm.com/course/foundations-of-quantum-error-correction>
- Companion (free): **Nielsen & Chuang**, ch 1–4, 8, 10 (read alongside)

4.4 Variational quantum algorithms — VQE, ADAPT-VQE, DMET, QAOA (~45 hrs)

You've shipped this; now own the math.

- **PennyLane Codebook — Variational Quantum Algorithms** (work the exercises) — <https://pennylane.ai/codebook>
- **PennyLane quantum-chemistry demos** (run *all* the derivations behind the code you've used) — https://pennylane.ai/qml/demos_quantum-chemistry
- Papers, re-derived: VQE (<https://www.nature.com/articles/ncomms5213>) · ADAPT-VQE (<https://www.nature.com/articles/s41467-019-10988-2>) · DMET-VQE / fragment methods (<https://arxiv.org/abs/2108.08987>) · QAOA (<https://arxiv.org/abs/1411.4028>) + Hadfield review (<https://arxiv.org/abs/1709.03489>) · Pauli decomposition + measurement grouping (shot-budget engineering, <https://arxiv.org/abs/1907.13117>)

Whiteboard checkpoint: derive the variational principle; walk VQE end-to-end (ansatz choice, parameter-shift rule, optimizer); explain barren plateaus mathematically + mitigations; why ADAPT-VQE is more parameter-efficient; when DMET fragmentation is worth the overhead.

4.5 Quantum machine learning theory (~42 hrs)

- **PennyLane QML topic page (Schuld-curated)** — the most honest overview in the field — <https://pennylane.ai/topics/quantum-machine-learning>
- **Maria Schuld — "Taking stock of QML: a critical perspective"** — the talk every senior should watch (tells you what's hype vs real) — https://www.youtube.com/results?search_query=maria+schuld+taking+stock+quantum+machine+learning
- Schuld & Petruccione, *Machine Learning with Quantum Computers* — ch 4–7 (kernels, variational, QNNs)
- Papers: "QML models are kernel methods" (<https://arxiv.org/abs/2101.11020>) · Barren plateaus (<https://www.nature.com/articles/s41467-018-07090-4>) · Expressibility & entangling capability (<https://arxiv.org/abs/1905.10876>) · "Is quantum advantage the right goal?" (<https://arxiv.org/abs/2203.01340>) · PQC encodings (<https://arxiv.org/abs/2008.08605>)
- Structured courses to pull lectures from: Christophe Péré (<https://github.com/Christophe-pere/QML-Course>) · Morten Hjorth-Jensen, Oslo (<https://github.com/CompPhysics/QuantumComputingMachineLearning>)

4.6 Quantum chemistry on quantum computers + capstone (~60 hrs)

- **Qiskit Global Summer School — recordings** (chemistry + variational tracks) — <https://www.youtube.com/@qiskit/playlists> — ~15 hrs
- **IBM Quantum Learning platform + Sample-based / Krylov Quantum Diagonalization (QDA)** (IBM's newest near-term technique) — <https://learning.quantum.ibm.com/> · <https://www.ibm.com/quantum/blog/iql-migration> — ~15 hrs
- **PHASE 4 CAPSTONE (~30 hrs)** — take one molecule from your Lilly work (keto-enol tautomerism or HATU coupling). Re-implement three ways — vanilla VQE + UCCSD, ADAPT-VQE, DMET-VQE — benchmark against classical CCSD/6-31G. The 4-page write-up defends *every* choice on theoretical grounds (ansatz, optimizer, measurement grouping, noise mitigation, basis set) and includes a barren-plateau analysis.

Phase 4 self-test: derive VQE from the variational principle; explain the parameter-shift rule and why it works; whiteboard a barren-plateau argument; VQE vs QAOA — when each, and when *no* quantum algorithm; honest take on where QML is overhyped vs promising (cite Schuld); walk through DMET fragmentation and the classical-quantum interface.

PHASE 5 — Full-Stack / Production AI Systems

~160 hrs · production breadth + systems-design readiness

Goal: Upgrade from "I built it" to "I can defend every architectural choice and design the next-gen version" across LLM systems, RAG, MLOps, distributed training, and serving.

5.1 LLM internals (~32 hrs)

- **Stanford CS336 — Language Modeling from Scratch** (currently the best LLM-systems course) — <https://stanford-cs336.github.io/spring2024/> · https://www.youtube.com/playlist?list=PLoROMvovd4rOY23Y0BoGoBGgQ1zmU_MT_
- **Hugging Face NLP course** (units new to you) — <https://huggingface.co/learn/nlp-course>

5.2 RAG at senior level (~28 hrs)

- **Hamel Husain — "What we learned from a year of building with LLMs"** (Parts I-II) — <https://www.oreilly.com/radar/what-we-learned-from-a-year-of-building-with-llms-part-i/>
- **Eugene Yan — LLM patterns** — <https://eugeneyan.com/writing/llm-patterns/>
- **Anthropic — contextual retrieval** — <https://www.anthropic.com/news/contextual-retrieval>
- Reranking, hybrid search, query rewriting, evals; **LlamaIndex** (<https://docs.llamaindex.ai/>) · **RAGAS** (<https://docs.ragas.io/>) · **DSPy** (<https://dspy.ai/>)

5.3 MLOps + ML systems design (~42 hrs)

- **Chip Huyen — Designing Machine Learning Systems** / interviews book — <https://huyenchip.com/ml-interviews-book/contents/8.1.1-ml-systems-design.html>
- **Stanford CS329S — ML Systems Design** — <https://stanford-cs329s.github.io/>
- **Made With ML** (hands-on) — <https://madewithml.com/> · **Full Stack Deep Learning** — <https://fullstackdeeplearning.com/course/2022/>

5.4 Distributed training + inference optimization (~25 hrs)

- **Sebastian Raschka** (distributed training + efficient fine-tuning) — <https://magazine.sebastianraschka.com/>
- **HF — efficient training on multiple GPUs** — https://huggingface.co/docs/transformers/en/perf_train_gpu_many
- **vLLM** (paper + run it) — <https://arxiv.org/abs/2309.06180> · quantization (GPTQ/AWQ/GGUF/FP8) · **ZeRO** — <https://arxiv.org/abs/1910.02054>

5.5 Eval / observability / safety + systems-design interview prep (~33 hrs)

- **Anthropic — Constitutional AI** — <https://www.anthropic.com/news/constitutional-ai> · **Eugene Yan — LLM evals** — <https://eugeneyan.com/writing/llm-evaluators/>
- Observability tool (LangSmith / Phoenix / Helicone) — run one on a real RAG system.
- **Systems-design interview prep** — Chip Huyen's book (<https://huyenchip.com/ml-interviews-book/>); practice on a whiteboard, recorded, self-critiqued: "RAG assistant for HCPs, 10K queries/day on 50K docs"; "brain-seg pipeline ingesting new-institution data weekly with domain-shift handling"; "retraining + monitoring for a drug-discovery digital twin."

PHASE 6 — Specialization + Integration + Interview Prep

~135 hrs · the culmination — ties imaging, quantum, and drug-discovery together

Goal: Close the geometric-deep-learning / molecular-AI gap that connects all your work, then integrate everything and become interview-ready as a senior engineer.

6.1 Geometric deep learning + molecular ML (~45 hrs)

- **Bronstein et al. — Geometric Deep Learning course** — <https://geometricdeeplearning.com/lectures/> · proto-book: <https://arxiv.org/abs/2104.13478> · Veličković GNN lectures: https://www.youtube.com/results?search_query=petar+velickovic+gnn+lectures
- **Molecular ML** — DeepChem (<https://deepchem.io/tutorials/>) · Therapeutics Data Commons (<https://tdcommons.ai/>) · SchNet (<https://arxiv.org/abs/1706.08566>) · DimeNet++ (<https://arxiv.org/abs/2011.14115>) · MolFormer (<https://arxiv.org/abs/2106.09553>) · equivariant nets, Cohen & Welling (<https://arxiv.org/abs/1602.07576>)

6.2 Protein structure + equivariance (~25 hrs)

Ties geometric DL, foundation models, biology, and quantum chemistry together.

- **AlphaFold 2** — AlQuraishi explainer (https://www.youtube.com/results?search_query=AlphaFold+2+explained+AlQuraishi) · paper (<https://www.nature.com/articles/s41586-021-03819-2>) · ESM/ESMFold · e3nn/EGNN

6.3 Integration project (~30 hrs) — your portfolio centerpiece

Something touching multiple domains — e.g., predict a molecular property with a hybrid quantum-classical model pretrained on a chemical foundation model, deployed as a monitored service. This is the artifact that demonstrates senior-level range.

6.4 Mock interviews + portfolio + writing (~35 hrs)

- **Recorded, reviewed:** 60-min ML systems design x several; 45-min segmentation deep-dive (defend loss/architecture choices); 45-min quantum (defend VQE/ADAPT/DMET to a skeptical chemist); 30-min behavioral (STAR, 4 stories prepped).
- **Read 1 paper/day** from your subfields; **write 2 short blog/LinkedIn posts** distilling something learned — writing for an audience exposes shaky understanding.

== BLOCK 2 — Breadth, GenAI & Big-Tech Readiness

mid-March → August 2027 · ~500 hrs + interviewing · ~30-34 hrs/week · Thread T (DSA + C++) continues underneath

PHASE 7 — Generative AI & Agentic AI

~150 hrs · Weeks 31-35 (~mid-Mar → mid-Apr 2027)

Goal: Move from "I use LLMs in a RAG app" to designing, adapting, and *orchestrating* modern GenAI and agentic systems — the fastest-moving, most in-demand area right now. Builds directly on Phase 5's LLM/RAG foundation.

7.1 Advanced LLMs — adaptation & inference (~40 hrs)

- Fine-tuning & PEFT: **LoRA** (<https://arxiv.org/abs/2106.09685>) · **QLoRA** (<https://arxiv.org/abs/2305.14314>); Sebastian Raschka's practical LoRA writeups (<https://magazine.sebastianraschka.com/>)
- Preference tuning: **InstructGPT / RLHF** (<https://arxiv.org/abs/2203.02155>) · **DPO** (<https://arxiv.org/abs/2305.18290>); Hugging Face **TRL** library
- **Hugging Face LLM course** (fine-tuning + advanced chapters) — <https://huggingface.co/learn/llm-course>
- **Chip Huyen — AI Engineering** (O'Reilly, 2025), the best single text on building GenAI products — read alongside the phase

7.2 Multimodal & vision-language models (~25 hrs) — also feeds Phase 8

- **CLIP** (<https://arxiv.org/abs/2103.00020>) · **LLaVA** (<https://arxiv.org/abs/2304.08485>) · **Flamingo** (<https://arxiv.org/abs/2204.14198>): how image encoders bolt onto LLMs, and where multimodal RAG fits.

7.3 Agents & agentic systems (~50 hrs) — the core of this phase

- **Hugging Face Agents Course** (free, certified; smolagents, LangGraph, LlamaIndex — hands-on) — <https://huggingface.co/learn/agents-course>
- **Hugging Face MCP Course** (Model Context Protocol, in theory + practice) — <https://huggingface.co/learn/mcp-course>
- **Anthropic — Building Effective Agents** (the clearest guide to agent design vs. over-engineering) — <https://www.anthropic.com/research/building-effective-agents>
- **Andrew Ng — agentic design patterns** (reflection, tool use, planning, multi-agent) via DeepLearning.AI short courses — <https://www.deeplearning.ai/short-courses/>
- **LangGraph** (stateful, multi-step orchestration) — <https://langchain-ai.github.io/langgraph/>
- Patterns to internalize: ReAct (reason+act), tool/function calling, reflection & self-critique, planning/decomposition, memory, multi-agent collaboration; plus **agent evaluation** (why it's hard, how to do it).

7.4 Capstone (~35 hrs) — build a real agent

Ship a non-trivial agentic system — e.g., a multi-step research/analysis agent over your domain (medical-imaging literature or drug-discovery data) with tool use, memory, retrieval, and an evaluation harness. Write up the architecture

and the failure modes you engineered around: portfolio piece *and* GenAI-systems-design interview story.

Phase 7 self-test: explain LoRA and why it's parameter-efficient; RLHF vs DPO; when an agent beats a single well-prompted call (and when it's needless complexity); walk the ReAct loop; how you'd evaluate an agent's reliability.

PHASE 8 — Computer Vision Expert

~200 hrs • Weeks 36-42 (~mid-Apr → early Jun 2027)

Goal: Round out from "deep-learning-for-medical-images" into a *broad* CV expert — classical & geometric vision, 3D, detection/tracking, generative vision — plus the deployment/optimization skills that put CV models onto real hardware (your integration need). Block 1's Phases 2-3 are the modern-DL half; this is the classical/geometric half plus the systems half, and it's where your C++ thread pays off.

8.1 Classical & geometric computer vision (~50 hrs) — the foundation most ML engineers lack

- **First Principles of Computer Vision — Shree Nayar (Columbia)** — the gold standard: image formation, cameras & optics, image processing, features & boundaries, 3D reconstruction (single & multi-view), perception. Free lectures + monographs — <https://fpcv.cs.columbia.edu/> (YouTube "First Principles of Computer Vision"; Coursera specialization: <https://www.coursera.org/specializations/firstprinciplesofcomputervision>)
- **Cyrill Stachniss — Photogrammetry & Mobile Sensing** (YouTube; multi-view geometry, SLAM, point clouds, rigorous) — <https://www.youtube.com/@CyrillStachniss>
- Own: camera models & calibration, homography, epipolar geometry, stereo, feature detection/matching (SIFT/ORB), optical flow, structure-from-motion, visual SLAM.
- Reference: Hartley & Zisserman, *Multiple View Geometry in Computer Vision*

8.2 Modern DL for vision — breadth (~35 hrs)

- **Michigan EECS 498 — Deep Learning for Computer Vision (Justin Johnson)** — the modern successor to CS231n; go broad on detection, segmentation, video, 3D, generative — <https://web.eecs.umich.edu/~justincj/teaching/eecs498/> · playlist: <https://www.youtube.com/playlist?list=PL5-TkQAFbzxjBHtZdVCWE0Zbhmg7r>
- Object detection (two-stage vs one-stage), instance/panoptic segmentation, pose estimation, multi-object tracking (build on your YOLO/DeepSORT experience), video understanding.

8.3 3D vision & neural rendering (~30 hrs)

- Depth estimation, point-cloud processing; **NeRF** (<https://arxiv.org/abs/2003.08934>) · **3D Gaussian Splatting** (<https://arxiv.org/abs/2308.04079>) — the current frontier of 3D reconstruction.

8.4 Deployment & optimization for hardware/edge (~70 hrs) — CV expert meets integration

- **OpenCV in C++** — classical CV pipelines at production speed — <https://docs.opencv.org/> · applied recipes: <https://pyimagesearch.com/>
- **On-device serving & optimization:** ONNX (<https://onnx.ai/>), **NVIDIA TensorRT** (<https://developer.nvidia.com/tensorrt>), quantization & pruning for CV models, real-time/edge inference tradeoffs.
- **CUDA basics** — enough to reason about GPU kernels and data movement for a vision pipeline (NVIDIA intro materials).
- Practical target: take one of your segmentation/detection models and get it running optimized (TensorRT/ONNX, quantized) behind a **C++ inference path** — a concrete integration artifact.

8.5 CV expert capstone (~15 hrs)

A geometric-CV or 3D project that isn't deep-learning-only (e.g., calibrate cameras and reconstruct a small scene via SfM, or a feature-based visual-odometry demo), deployed with a C++/optimized inference component. Demonstrates the classical + systems range that separates a CV *expert* from a CNN *user*.

Phase 8 self-test: derive the pinhole camera model and the epipolar constraint; explain SIFT and why scale/rotation invariance matters; two-stage vs one-stage detectors; what TensorRT actually does to speed up inference; how you'd port a PyTorch CV model to a real-time C++ pipeline.

PHASE 9 — Big-Tech Interview Crescendo

~150 hrs · Weeks 43-47 (~Jun → mid-Jul 2027), then maintenance through interview season

Goal: Peak the interview-specific skills — DSA under time pressure, system design, behavioral — right as you start interviewing at Google-caliber companies. This is where the year-long Thread T converts into performance: you're sharpening a reflex you already have, not starting cold.

9.1 DSA — interview-grade (~70 hrs) — the crescendo of Thread T

- Timed practice: **NeetCode 150** → **Blind 75** review, then company-tagged sets on **LeetCode** — <https://neetcode.io/> · <https://leetcode.com/>
- Pattern mastery under pressure: **DesignGurus — Grokking the Coding Interview** — <https://www.designgurus.io/>
- Do problems *out loud*, on a blank editor, timed to ~35 min, then review optimal solutions — ~4-6/day this phase.

9.2 System design (~55 hrs) — distinct from the ML-systems-design in Phase 5

- **ByteByteGo / Alex Xu — System Design Interview** (vol 1 & 2) + the channel — <https://bytebytego.com/> · <https://www.youtube.com/@ByteByteGo>
- **Designing Data-Intensive Applications** — Martin Kleppmann (the depth behind the interview: replication, partitioning, consistency, batch/stream) — book
- **Gaurav Sen — System Design** (YouTube; worked walkthroughs) — <https://www.youtube.com/@gkcs>
- Optional depth: **MIT 6.824 — Distributed Systems** — <https://pdos.csail.mit.edu/6.824/>
- Whiteboard 8-10 classic prompts (URL shortener, news feed, rate limiter, chat, video platform, search) *and* ML-flavored ones (recommendation service, feature store, model-serving platform) — recorded, self-critiqued.

9.3 Behavioral, "Googleness" & low-level design (~25 hrs)

- Structure ~6-8 STAR stories (impact, conflict, failure, leadership, ambiguity) mapped to Google's leadership signals; practice out loud.
- **Object-oriented / low-level design** (parking lot, elevator, rate limiter as objects) — common in some loops.
- Mock interviews across all three tracks, recorded and reviewed; **Pramp / interviewing.io** for live peer mocks.

Phase 9 self-test: a random LeetCode hard in ~45 min with clean code; a full system-design whiteboard in 45 min with capacity estimates and articulated tradeoffs; four STAR stories delivered crisply from memory.

(Weeks 48-55, ~mid-Jul → Aug 2027: active interviewing. Keep DSA and system design warm with ~5 hrs/week maintenance; use the buffer for on-site loops, take-homes, and rest between them.)

Cross-cutting habits (every week, no exceptions)

1. **Paper-of-the-week** — one paper, deeply read, derivations redone on paper. Don't break the streak.
2. **Whiteboard Friday** — one concept whiteboarded without notes; photograph it. These become your interview-prep deck.
3. **Re-implementation Saturday** — one concept from scratch in <200 lines. Repo: [weekly-rebuild/wk01-linalg](#), etc.
4. **Sunday writeup** — a 1-page note explaining the week's deepest concept to a hypothetical junior. Teaching is the senior-level forcing function.
5. **Senior-engineer "war chest"** — one Obsidian/Notion doc collecting derivations, paper one-liners, gotchas, anti-patterns. It becomes your interview safety net.

Resource index (quick lookup)

Brush-up (Phase 0) — StatQuest playlists <https://www.youtube.com/@statquest/playlists> · StatQuest video index <https://statquest.org/> · 3B1B Neural Networks https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

Foundations math — 3B1B LA https://www.youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab · 3B1B Calc <https://www.youtube.com/playlist?list=PLZHQObOWTQDMSr9K-rj53DwVRMYO3t5Yr> · MIT 18.06 <https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/> · MIT 18.065 <https://ocw.mit.edu/courses/18-065-matrix-methods-in-data-analysis-signal-processing-and-machine-learning-spring-2018/> · Stat 110 <https://projects.iq.harvard.edu/stat110/youtube> · MIT 6.041 <https://ocw.mit.edu/courses/res-6-012-introduction-to-probability-spring-2018/> · Matrix Cookbook <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf> · Matrix Calculus <https://explained.ai/matrix-calculus/> · Karpathy Zero to Hero <https://www.youtube.com/playlist?list=PLAqhlrjkbW123v9cThsA9GvCAUhRvKZ> · hub <https://karpathy.ai/zero-to-hero>

hero.html

Later-threaded math — Boyd <https://www.youtube.com/playlist?list=PL3940DD956CDF0622> · <https://web.stanford.edu/~boyd/cvxbook/> · Ruder <https://www.ruder.io/optimizing-gradient-descent/> · MacKay <https://www.inference.org.uk/itila/book.html> · Group theory (search) https://www.youtube.com/results?search_query=group+theory+for+physicists+lectures · GDL lectures <https://geometricdeeplearning.com/lectures/>

Medical imaging — MRI Q&A <https://mrquestions.com/index.html> · OHBM <https://www.youtube.com/@ohbmonline/playlists> · Callaghan https://www.youtube.com/playlist?list=PLqEMVgX2VgZcG_xR1QRdsuc-X_zEgMx-Z · iBiology <https://www.youtube.com/playlist?list=PLF513KEDjY9YBNhwMfX6jMI3PlpdW9TMP> · AI for Medical Diagnosis <https://www.coursera.org/learn/ai-for-medical-diagnosis> · DigitalSreeni <https://www.youtube.com/@DigitalSreeni/playlists> · code https://github.com/bnsreeni/python_for_microscopists · Conv arithmetic <https://arxiv.org/abs/1603.07285> · https://github.com/vdumoulin/conv_arithmetic · MONAI tutorials <https://github.com/Project-MONAI/tutorials> · MONAI YouTube <https://www.youtube.com/@projectmonai/videos> · nnU-Net <https://github.com/MIC-DKFZ/nnUNet> · TorchIO <https://torchio.readthedocs.io/> · Stanford AIMI <https://aimi.stanford.edu/education/educational-resources> · CS231n <https://www.youtube.com/playlist?list=PL3FW7Lu3i5jvHM8ljYj-zLfQRF3EO8sYv> · <https://cs231n.github.io/> · Metrics Reloaded <https://www.nature.com/articles/s41592-023-02151-z> *Segmentation papers*: U-Net <https://arxiv.org/abs/1505.04597> · V-Net <https://arxiv.org/abs/1606.04797> · Attention U-Net <https://arxiv.org/abs/1804.03999> · nnU-Net <https://www.nature.com/articles/s41592-020-01008-z> · Loss survey <https://arxiv.org/abs/2006.14822> · Generalized Dice <https://arxiv.org/abs/1707.03237> · Focal Tversky <https://arxiv.org/abs/1810.07842> · Boundary loss <https://arxiv.org/abs/1812.07032> · ViT <https://arxiv.org/abs/2010.11929> · TransUNet <https://arxiv.org/abs/2102.04306> · Swin-UNet <https://arxiv.org/abs/2105.05537> · UNETR <https://arxiv.org/abs/2103.10504> · Swin UNETR <https://arxiv.org/abs/2201.01266> · SAM <https://arxiv.org/abs/2304.02643> · MedSAM <https://www.nature.com/articles/s41467-024-44824-z> · MedNeXt <https://arxiv.org/abs/2303.09975>

Deep learning core — CS25 <https://web.stanford.edu/class/cs25/> · https://www.youtube.com/playlist?list=PLoROMvodv4rNijRchCzutFw5ltR_Z27CM · CS336 <https://stanford-cs336.github.io/spring2024/> · https://www.youtube.com/playlist?list=PLoROMvodv4rOY23Y0BoGoBGgQ1zmU_MT_ · CS236 <https://www.youtube.com/playlist?list=PLoROMvodv4rPOWA-omMM6STXaWW4FvjT8> · Lilian Weng <https://lilianweng.github.io/> · Annotated Transformer <http://nlp.seas.harvard.edu/annotated-transformer/> · HF NLP <https://huggingface.co/learn/nlp-course> · HF CV <https://huggingface.co/learn/computer-vision-course> · HF Diffusion <https://huggingface.co/learn/diffusion-course> *Papers*: RoPE <https://arxiv.org/abs/2104.09864> · RMSNorm <https://arxiv.org/abs/1910.07467> · FlashAttention <https://arxiv.org/abs/2205.14135> · GQA <https://arxiv.org/abs/2305.13245> · SwiGLU <https://arxiv.org/abs/2002.05202> · Switch/MoE <https://arxiv.org/abs/2101.03961> · ConvNeXt <https://arxiv.org/abs/2201.03545> · DINOv2 <https://arxiv.org/abs/2304.07193> · MAE <https://arxiv.org/abs/2111.06377> · DDPM <https://arxiv.org/abs/2006.11239> · Score-based (blog) <https://yang-song.net/blog/2021/score/> · Diffusion (blog) <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>

Quantum — IBM Quantum Learning <https://learning.quantum.ibm.com/> · Watrous YouTube <https://www.youtube.com/playlist?list=PLOFEBzvs-VvrXTMy5Y2lqmSaUjfnhvBHR> · Basics <https://learning.quantum.ibm.com/course/basics-of-quantum-information> · Algorithms <https://learning.quantum.ibm.com/course/fundamentals-of-quantum-algorithms> · General Formulation <https://learning.quantum.ibm.com/course/general-formulation-of-quantum-information> · Error Correction <https://learning.quantum.ibm.com/course/foundations-of-quantum-error-correction> · Qiskit YouTube <https://www.youtube.com/@qiskit/playlists> · QDA blog <https://www.ibm.com/quantum/blog/iql-migration> · PennyLane codebook <https://pennylane.ai/codebook> · PennyLane QML topic <https://pennylane.ai/topics/quantum-machine-learning> · PennyLane chemistry https://pennylane.ai/qml/demos_quantum-chemistry · MIT 8.04 <https://ocw.mit.edu/courses/8-04-quantum-physics-i-spring-2013/> · Quantum Country <https://quantum.country/qcvc> · Schuller https://www.youtube.com/playlist?list=PLPH7f_7ZlzxTi6kS4vCmv4ZKm9u8g5yic · Péré <https://github.com/Christophe-pere/QML-Course> · Hjorth-Jensen <https://github.com/CompPhysics/QuantumComputingMachineLearning> *Papers*: VQE <https://www.nature.com/articles/ncomms5213> · ADAPT-VQE <https://www.nature.com/articles/s41467-019-10988-2> · DMET-VQE <https://arxiv.org/abs/2108.08987> · QAOA <https://arxiv.org/abs/1411.4028> · Hadfield <https://arxiv.org/abs/1709.03489> · QML=kernels <https://arxiv.org/abs/2101.11020> · Barren plateaus <https://www.nature.com/articles/s41467-018-07090-4> · Expressibility <https://arxiv.org/abs/1905.10876> · Advantage-goal <https://arxiv.org/abs/2203.01340> · PQC encodings <https://arxiv.org/abs/2008.08605> · Pauli grouping <https://arxiv.org/abs/1907.13117>

Production / MLOps — CS329S <https://stanford-cs329s.github.io/> · Made With ML <https://madewithml.com/> · Full Stack DL <https://fullstackdeeplearning.com/course/2022/> · Chip Huyen <https://huyenchip.com/ml-interviews-book/> · Hamel Husain <https://www.oreilly.com/radar/what-we-learned-from-a-year-of-building-with-llms-part-i/> · Eugene Yan patterns <https://eugeneyan.com/writing/llm-patterns/> · Eugene Yan evals <https://eugeneyan.com/writing/llm-evaluators/> · Anthropic contextual retrieval <https://www.anthropic.com/news/contextual-retrieval> · Anthropic Constitutional AI <https://www.anthropic.com/news/constitutional-ai> · LlamaIndex <https://docs.llamaindex.ai/> · RAGAS <https://docs.ragas.io/> · DSPy <https://ds.py.ai/> · Raschka <https://magazine.sebastianraschka.com/> · HF multi-GPU https://huggingface.co/docs/transformers/en/perf_train_gpu_many · vLLM <https://arxiv.org/abs/2309.06180> · ZeRO <https://arxiv.org/abs/1910.02054>

DSA & C++ (Thread T) — leetcode <https://www.leetcode.com/> · The Cherno <https://www.youtube.com/@TheCherno> · cppreference <https://en.cppreference.com/> · NeetCode <https://neetcode.io/> · LeetCode <https://leetcode.com/> · MIT 6.006 <https://www.6.006.mit.edu/>

<https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> · Abdul Bari https://www.youtube.com/@abdul_bari

GenAI & Agents (Phase 7) — HF LLM course <https://huggingface.co/learn/llm-course> · HF Agents course <https://huggingface.co/learn/agents-course> · HF MCP course <https://huggingface.co/learn/mcp-course> · Anthropic Building Effective Agents <https://www.anthropic.com/research/building-effective-agents> · DeepLearning.AI short courses <https://www.deeplearning.ai/short-courses/> · LangGraph <https://langchain-ai.github.io/langgraph/> · LoRA <https://arxiv.org/abs/2106.09685> · QLoRA <https://arxiv.org/abs/2305.14314> · InstructGPT/RLHF <https://arxiv.org/abs/2203.02155> · DPO <https://arxiv.org/abs/2305.18290> · CLIP <https://arxiv.org/abs/2103.00020> · LLaVA <https://arxiv.org/abs/2304.08485> · Flamingo <https://arxiv.org/abs/2204.14198>

Computer Vision expert (Phase 8) — First Principles of CV <https://fpcv.cs.columbia.edu/> · FPCV Coursera <https://www.coursera.org/specializations/firstprinciplesofcomputervision> · Cyrill Stachniss <https://www.youtube.com/@CyrillStachniss> · Michigan EECS 498 <https://web.eecs.umich.edu/~justincj/teaching/eecs498/> · playlist <https://www.youtube.com/playlist?list=PL5-TkQAfAZFbzxjBHTzdVCWE0Zbhong7r> · NeRF <https://arxiv.org/abs/2003.08934> · 3D Gaussian Splatting <https://arxiv.org/abs/2308.04079> · OpenCV <https://docs.opencv.org/> · PyImageSearch <https://pyimagesearch.com/> · ONNX <https://onnx.ai/> · TensorRT <https://developer.nvidia.com/tensorrt>

Big-tech interview (Phase 9) — NeetCode <https://neetcode.io/> · LeetCode <https://leetcode.com/> · DesignGurus <https://www.designgurus.io/> · ByteByteGo <https://bytebytego.com/> · ByteByteGo YouTube <https://www.youtube.com/@ByteByteGo> · Gaurav Sen <https://www.youtube.com/@gkcs> · MIT 6.824 <https://pdos.csail.mit.edu/6.824/>

Drug discovery / Geometric DL — Bronstein GDL <https://geometricdeeplearning.com/lectures/> · proto-book <https://arxiv.org/abs/2104.13478> · DeepChem <https://deepchem.io/tutorials/> · TDC <https://tdcommons.ai/> · SchNet <https://arxiv.org/abs/1706.08566> · DimeNet++ <https://arxiv.org/abs/2011.14115> · MolFormer <https://arxiv.org/abs/2106.09553> · Equivariant nets <https://arxiv.org/abs/1602.07576> · AlphaFold 2 <https://www.nature.com/articles/s41586-021-03819-2>

A note on what you're taking on

This is now a ~13-month, ~1,730-hour program on top of a full-time senior role — genuinely ambitious, and only sustainable if you treat it as a marathon. A few anchors: the **self-tests are the real gates**, not the calendar dates, which are planning aids you can flex; the **DSA/C++ thread is your shock absorber** — dial it to 2–3 hrs/week whenever a week gets heavy, since consistency beats volume for muscle memory; and **protect one true rest block a week** plus the buffer weeks, because burnout in month five costs more than any schedule it appears to save. Re-activating rusty fundamentals and re-deriving what you already use will feel like going backwards at first — it isn't; it's exactly what turns "I use this" into "I can teach this to a panel." If the calendar and your bandwidth ever truly collide, tell me and I'll rebalance. Adjust as you go: it's a scaffold for durable mastery, not a race to the bottom of your energy.